National University of Singapore
School of Computing
CS3244: Machine Learning
Solution to Tutorial 8 (1 week)

Neural Networks and Deep Learning

Readings: LFD, dynamic e-Chapter 7

1. **Combining perceptrons.** (Adapted from LFD Exercise 7.1, 7.3, 7.4) On Slide 4 of the lecture notes, we have constructed the XOR function from two simple perceptrons h_1 and h_2. Let's extend this to a more complicated function f shown below.

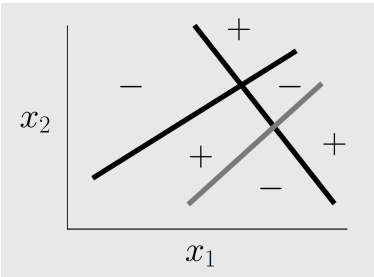


Figure 1: Target function f.

(a) Given the three perceptron components h_1, h_2 and h_3,

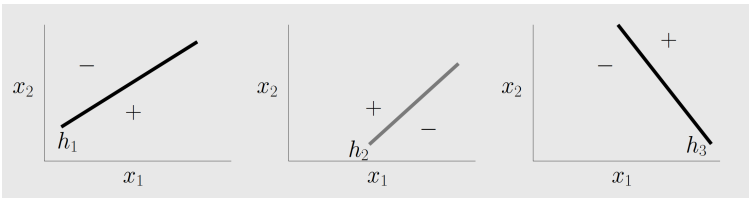
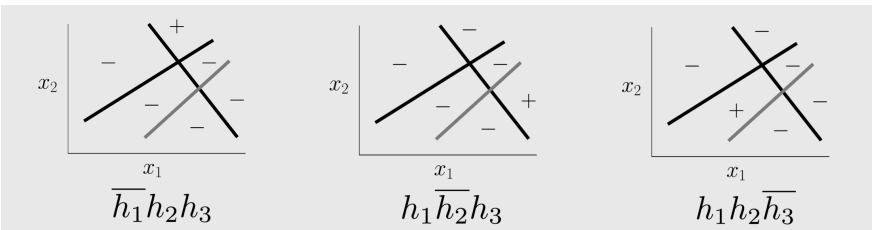


Figure 2: The three perceptron components of f.

verify that

$$f = \overline{h_1}h_2h_3 + h_1\overline{h_2}h_3 + h_1h_2\overline{h_3}.$$

The three boolean expressions corresponding to $\overline{h_1}h_2h_3$, $h_1\overline{h_2}h_3$ and $h_1h_2\overline{h_3}$ are illustrated in the following:



The addition in the equation for f is the boolean OR. By taking the OR of the three components, we obtain f. Note that each of the boolean terms corresponds to a region of

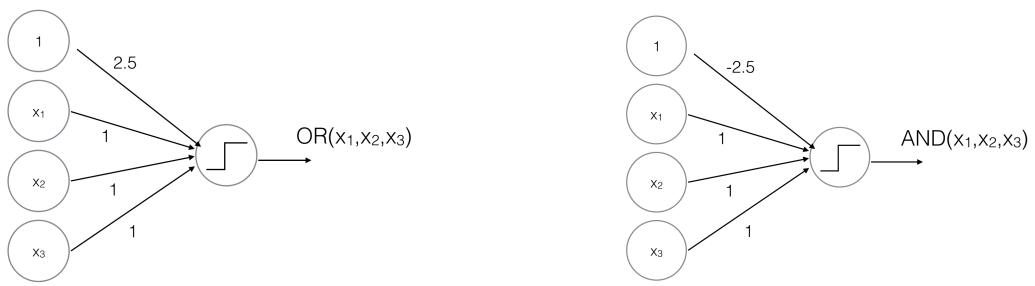
f where the target output is positive. If a certain region exhibits a $+$ in any of $\overline{h_1}h_2h_3$, $h_1\overline{h_2}h_3$ or $h_1h_2\overline{h_3}$, f will be $+$.

(b) Is there a systematic way to derive the boolean formula for f given its perceptron components? Hint: f given in (a) is of the disjunctive normal form (an OR of ANDs), what do each of the three terms in f correspond to?

As explained in (a), we can consider the regions of f where it is positive. For each of the positive regions, we derive a boolean expression which is an AND of the perceptron components (eg. $h_1, h_2 \cdots$) or their negations (eg. $\overline{h_1}, \overline{h_2} \cdots$). The target function f will then be an OR of the expressions for the positive regions.

(c) Construct the 3-layer multilayer perceptron for f, following the XOR example in the lecture notes.

Since there are three perceptron components to work with, we first define a graph representation for OR and AND with 3 inputs, as shown below. Note the difference with the 2-input OR, AND gates covered in the lecture notes. All inputs are ± 1. Check with a truth table that the weights and biases make sense.

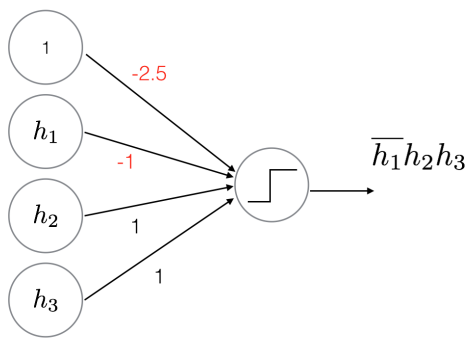


(a) Boolean OR.

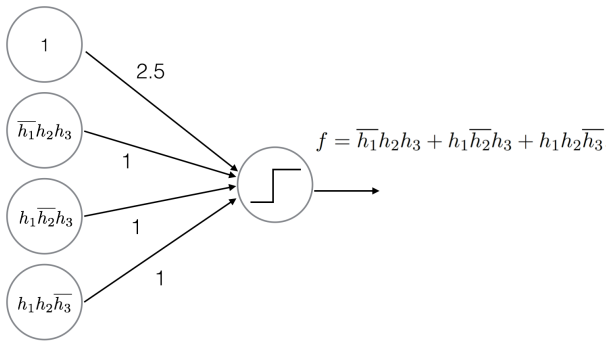
(b) Boolean AND.

Figure 3: Neural network for 3-input OR and AND.

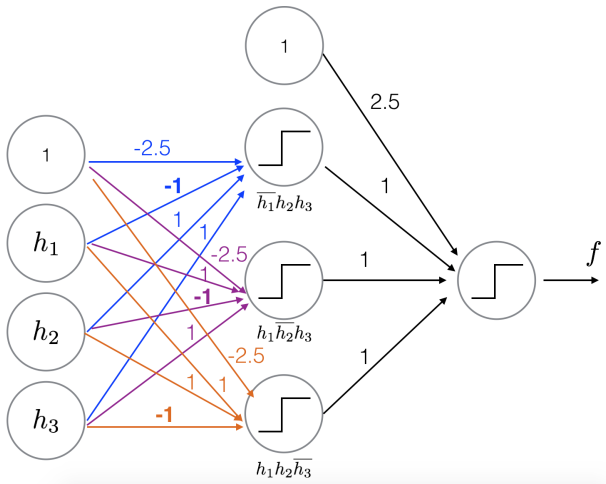
First, we express each of the positive regions as an AND of the three perceptron components. The figure below shows the graph for $\overline{h_1}h_2h_3$:



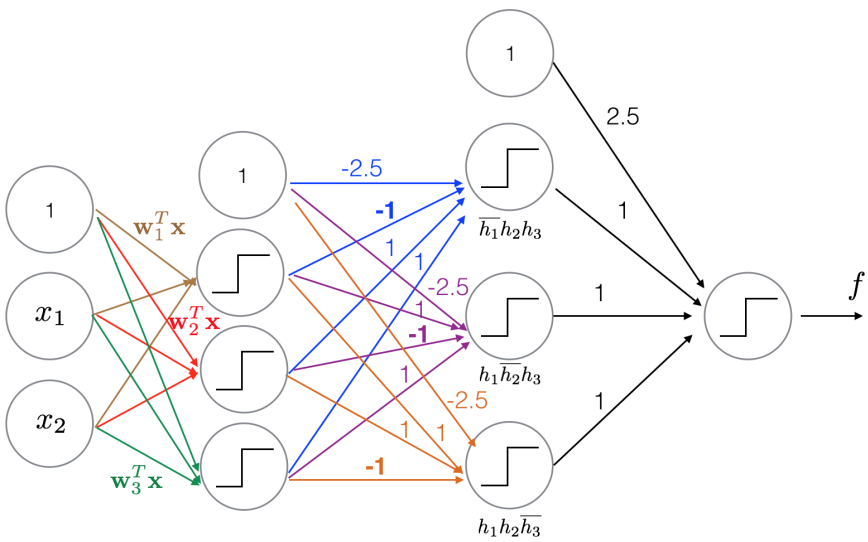
The same is done for $h_1\overline{h_2}h_3$ and $h_1h_2\overline{h_3}$. Once we have the graphs for the three positive regions, we obtain f by combining their outputs with an OR gate:



Combining all components, the graph looks like this:



This is not the end; we have to obtain the full 3-layered MLP from the inputs x_1 and x_2. Each of the perceptron components h_1, h_2, h_3 can be further expanded using $h_i = \text{sign}(\mathbf{w}_i^T\mathbf{x})$.



(d) Using your findings in (b) and (c), provide informal arguments for the following theorem: Any decision function whose ± 1 regions are defined in terms of the regions created by a set of hyperplanes can be implemented by a 3-layer perceptron.

Starting from the positive regions and linear perceptron components, we can construct an AND graph for each of the positive regions from knowing which side of the decision boundaries the region is on. The final target function will then be an OR gate connecting the graphs of the positive regions. This procedure can be done for any decision function (of any dimension) with well-defined ± 1 regions bounded by linear hyperplanes.

The result is a 3-layer MLP. The first layer constructs the linear hyperplanes from input data, the second layer obtains the positive regions while the last layer concatenates them with an OR gate to get the target function f. Depending on the complexity of f, the number of nodes in the intermediate layers may vary.

(e) Can the following function in Figure 4 be expressed with a 3-layer perceptron?

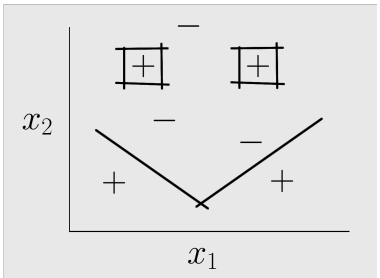
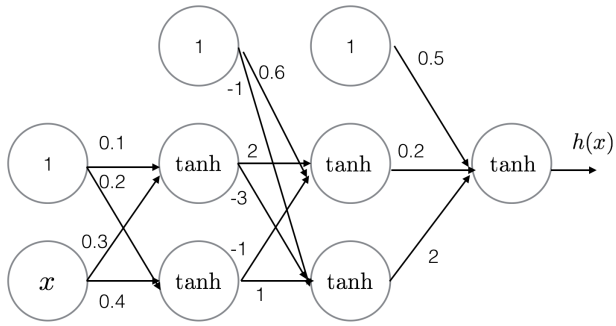


Figure 4: Function for 1(e).

Yes, as each ± 1 region is clearly defined by the hyperplanes given in the figure. We can derive f from the method explained in (b).

2. **Backpropagation algorithm.** (Adapted from LFD Example 7.1) For the following 3-layer neural network, there is a single data input x and output label y.



The weight matrices are:

$$W^{(1)} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}; W^{(2)} = \begin{bmatrix} 0.6 & -1 \\ 2 & -3 \\ -1 & 1 \end{bmatrix}; W^{(3)} = \begin{bmatrix} 0.5 \\ 0.2 \\ 2 \end{bmatrix}$$

(a) For the data point $x = 2, y = 1$, perform forward propagation to obtain the final output $\mathbf{x}^{(3)}$.

$$\mathbf{x}^{(0)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}; \mathbf{s}^{(1)} = \begin{bmatrix} 0.7 \\ 1 \end{bmatrix}; \mathbf{x}^{(1)} = \begin{bmatrix} 1 \\ 0.604 \\ 0.762 \end{bmatrix}; \mathbf{s}^{(2)} = \begin{bmatrix} 1.05 \\ -2.05 \end{bmatrix}; \mathbf{x}^{(2)} = \begin{bmatrix} 1 \\ 0.781 \\ -0.967 \end{bmatrix}$$

$$\mathbf{s}^{(3)} = \begin{bmatrix} -1.28 \end{bmatrix}; \mathbf{x}^{(3)} = \begin{bmatrix} -0.856 \end{bmatrix}$$

(b) Perform backpropagation to obtain the errors at each layer (ie. $\boldsymbol{\delta}^{(3)}, \boldsymbol{\delta}^{(2)}, \boldsymbol{\delta}^{(1)}$).

$$\boldsymbol{\delta}^{(3)} = \begin{bmatrix} -0.991 \end{bmatrix}; \boldsymbol{\delta}^{(2)} = \begin{bmatrix} -0.0774 \\ -0.127 \end{bmatrix}; \boldsymbol{\delta}^{(1)} = \begin{bmatrix} 0.143 \\ -0.0207 \end{bmatrix}$$

(c) Obtain the partial derivatives of the error with respect to the weights (ie. $\frac{\partial e}{\partial W^{(1)}}, \frac{\partial e}{\partial W^{(2)}}, \frac{\partial e}{\partial W^{(3)}}$).

$$\frac{\partial e}{\partial W^{(1)}} = \mathbf{x}^{(0)} \left(\boldsymbol{\delta}^{(1)}\right)^T = \begin{bmatrix} 0.143 & -0.0207 \\ 0.286 & -0.0415 \end{bmatrix}$$

$$\frac{\partial e}{\partial W^{(2)}} = \begin{bmatrix} -0.0774 & -0.127 \\ -0.0468 & -0.0766 \\ -0.0590 & -0.0966 \end{bmatrix}$$

$$\frac{\partial e}{\partial W^{(3)}} = \begin{bmatrix} -0.991 \\ -0.774 \\ 0.959 \end{bmatrix}$$

3. **Regularization with weight elimination.** (Adapted from LFD Exercise 7.11) For soft weight elimination, the regularization term is

$$\Omega(\mathbf{w}) = \sum_{i,j,l} \frac{\left(w_{ij}^{(l)}\right)^2}{1 + \left(w_{ij}^{(l)}\right)^2}$$

Hence, the combined augmented error of the network is

$$E_{\text{aug}}(\mathbf{w}) = E_{\text{in}}(\mathbf{w}) + \frac{\lambda}{N} \sum_{i,j,l} \frac{\left(w_{ij}^{(l)}\right)^2}{1 + \left(w_{ij}^{(l)}\right)^2}$$

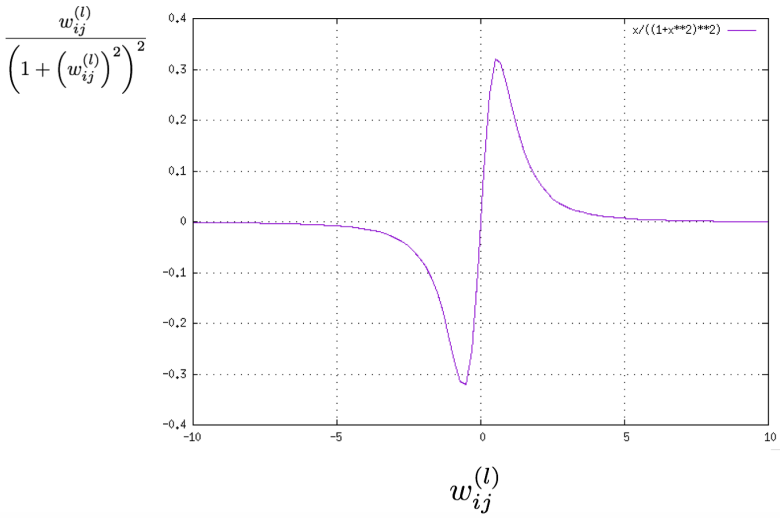
(a) By taking derivatives with respect to the network weights, show that

$$\frac{\partial E_{\text{aug}}}{\partial w_{ij}^{(l)}} = \frac{\partial E_{\text{in}}}{\partial w_{ij}^{(l)}} + \frac{2\lambda}{N} \frac{w_{ij}^{(l)}}{\left(1 + \left(w_{ij}^{(l)}\right)^2\right)^2}$$

First observe that for the weight elimination term, for the derivative with respect to $w_{ij}^{(l)}$, we only need to care about the relevant l^{th}, i^{th}, j^{th} term in the summation (the rest of the terms are zero after differentiation). We then proceed with the quotient rule for differentiation.

$$\begin{aligned}
\frac{\partial E_{\text{aug}}}{\partial w_{ij}^{(l)}} &= \frac{\partial E_{\text{in}}}{\partial w_{ij}^{(l)}} + \frac{\lambda}{N} \frac{2w_{ij}^{(l)}\left(1 + \left(w_{ij}^{(l)}\right)^2\right) - 2w_{ij}^{(l)}\left(w_{ij}^{(l)}\right)^2}{\left(1 + \left(w_{ij}^{(l)}\right)^2\right)^2} \qquad (1) \\
&= \frac{\partial E_{\text{in}}}{\partial w_{ij}^{(l)}} + \frac{2\lambda}{N} \frac{w_{ij}^{(l)}\left[1 + \left(w_{ij}^{(l)}\right)^2 - \left(w_{ij}^{(l)}\right)^2\right]}{\left(1 + \left(w_{ij}^{(l)}\right)^2\right)^2} \\
&= \frac{\partial E_{\text{in}}}{\partial w_{ij}^{(l)}} + \frac{2\lambda}{N} \frac{w_{ij}^{(l)}}{\left(1 + \left(w_{ij}^{(l)}\right)^2\right)^2}
\end{aligned}$$

(b) From the above, argue that weight elimination causes small weights to decay faster than larger weights.



By plotting the gradient of the weight elimination term on the right against $w_{ij}^{(l)}$, we notice how the gradient term changes as weight magnitude varies. The y-value represents the speed of weight decay (or weight increase for when weight is negative). Focusing just on the right side where weight is positive, there is a peak when the weight is small, beyond that peak, as weight increases, the speed of decay decreases. The converse is true for when weight is negative.

The two peaks may seem redundant, but it is a consequence of the function needing to change sign when weight changes sign and yet maintaining the characteristic that generally, large weights decay faster. This can be avoided if we allow discontinuities in the function (eg. like the $1/x$ graph), but remember that we need our loss function to be continuous and differentiable.